Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 01 Notes
Nested Classes (Types + Practical Use-Cases)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 How to use these notes (self-study) **2**

3 Prerequisites **2**

4 Learning Outcomes **2**

5 Lecture Overview **3**

6 Prerequisite Bridge **3**

7 Key Terms (Quick) **3**

8 Detailed Notes **3**

8.1 Nested classes at a glance . . . 3

8.2 Motivation: scope and encapsulation . . . 4

8.3 Types of nested classes (with intuition) . . . 4

8.3.1 1) Static nested class . . . 4

8.3.2 2) Inner class (non-static member class) . . . 4

8.3.3 3) Local class (inside a method) . . . 5

8.3.4 4) Anonymous class . . . 5

8.4 Access rules (important and often surprising) . . . 5

8.5 Common mistakes (and quick fixes) . . . 6

9 Running Project Connection **6**

10 Common Mistakes and Confusions **6**

11 Worked Example (tutorial): one outer type using all 4 kinds **6**

12 Demo / Observation Task **8**

13 Practice Check (with brief solutions) **8**

14 Self-Study Practice (with hints) **9**

15 Summary (1 minute) **9**

16 Further Study **9**

17 Exit Questions (with sample answers) **9**

1 Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O
- **Main new ideas:** Nested classes at a glance, Motivation: scope and encapsulation, Types of nested classes (with intuition)
- **Course thread:** Use Nested Classes (Types + Use-cases) to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

2 How to use these notes (self-study)

Read in order. Try the tiny code snippets mentally first, then run the worked example. For each section, answer the checkpoint questions before you move on.

3 Prerequisites

You should already be comfortable with:

- classes/objects and access modifiers (`private/public`),
- interfaces and implementations,
- basic method calls and constructors.

4 Learning Outcomes

After this lecture, you should be able to:

- Explain why nested classes improve encapsulation in some designs.
- Differentiate static nested, inner, local, and anonymous classes.
- Recognize use cases: helpers and callbacks/listeners (Swing later).

5 Lecture Overview

A **nested class** is a class declared **inside** another class, or inside a method. Java supports nested classes because sometimes a type is only meaningful within a bigger type. Nested classes let you:

- keep helpers **close** to where they are used (readability),
- hide implementation details (encapsulation),
- implement small callbacks/listeners without creating many separate files.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

6 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Class structure**: Nested classes still depend on the class-object model from Unit 02. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Interfaces and callbacks**: Anonymous and local classes make more sense once contract-based behavior is familiar. Main reference: `Unit 02 / Lecture 05 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

7 Key Terms (Quick)

- **Static nested class**: nested class with `static`; no implicit outer instance.
- **Inner class**: non-static member class; has implicit reference to outer object.
- **Local class**: class declared inside a method.
- **Anonymous class**: unnamed local class used inline to implement an interface/extend a class.
- **Callback**: passing code/behavior to run later (often via interfaces).
- **Effectively final**: a local variable that is not reassigned after initialization.
- **Outer instance**: the object of the outer class that an inner class "belongs to".

8 Detailed Notes

8.1 Nested classes at a glance

Use this table when you are unsure which kind you need:

Type	Needs outer object?	Typical use-case
Static nested	No	helper type tied to the outer concept, such as a builder or node
Inner (member)	Yes	helper that needs outer state, such as an iterator or controller
Local (in method)	No (but can capture)	short named helper used inside one method only
Anonymous	No (but can capture)	one-time callback or listener implementation

8.2 Motivation: scope and encapsulation

Sometimes a helper type is only meaningful inside one class. If you declare it as a top-level class, it becomes visible everywhere and increases the public "API surface". By nesting it, you communicate: *this helper belongs here.*

8.3 Types of nested classes (with intuition)

8.3.1 1) Static nested class

- Declared with `static`.
- Does **not** need an outer object to be created.
- Can only directly access **static** members of the outer class.

Use-case: helper/utility class tied to a concept (e.g., `Map.Entry`, `Builder`, `Node`).

Mini example (validator helper).

```java
class BankAccount {
    private double balance;

    static class Validator {
        static void requirePositive(double amount) {
            if (amount <= 0) throw new IllegalArgumentException("amount");
        }
    }

    void deposit(double amount) {
        Validator.requirePositive(amount);
        balance += amount;
    }
}
```

Notice: the validator does not need an outer object, so it is static.

8.3.2 2) Inner class (non-static member class)

- Has an implicit reference to the outer object.
- Can access outer instance fields/methods directly.

Use-case: iterators or helpers that need access to the outer object's state.

How to create an inner class object (syntax). An inner class instance is tied to an outer instance:

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
```

Important design note. Because an inner class carries an outer reference, be careful when the inner object may live longer than the outer object (can keep it in memory longer than expected). If you do not need outer state, prefer a static nested class.

8.3.3 3) Local class (inside a method)

- Exists only within the method scope.
- Can access local variables only if they are **effectively final**.

Use-case: small helper that is too specific to become a member class.

Why effectively final? When a local/anonymous class uses a local variable, the compiler "captures" its value. If that value could change later, the captured value would be confusing/unsafe. So Java requires the variable not to be reassigned.

8.3.4 4) Anonymous class

- Created inline with `new Interface() { ... }`.
- Useful for one-time implementations (callbacks/listeners).

In modern Java, lambdas often replace anonymous classes for functional interfaces (Unit 04), but anonymous classes are still important to understand because:

- older code uses them frequently,
- you can create an anonymous class even for non-functional interfaces and abstract classes.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

8.4 Access rules (important and often surprising)

- The outer class can access private members of the nested class.
- The nested class can access private members of the outer class.

This is allowed because the Java compiler generates the necessary bridge code. So `private` is still respected at the source-code level, but nested/outer types are considered closely related.

8.5 Common mistakes (and quick fixes)

1. **Mistake:** Trying to create an inner class without an outer object.

   Fix: create an outer instance first: `outer.new Inner()`.

2. **Mistake:** Local/anonymous class uses a variable that you later modify.

   Fix: keep the variable effectively final (do not reassign), or move state into a field.

3. **Mistake:** Choosing an inner class when a static nested class is enough.

   Fix: if you do not need outer instance data, make it `static`.

9 Running Project Connection

Use Nested Classes (Types + Use-cases) to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

10 Common Mistakes and Confusions

- Confusing static nested classes with true inner classes.
- Overusing anonymous classes when the logic is too large to read.
- Scattering tiny helper types across many files without need.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

11 Worked Example (tutorial): one outer type using all 4 kinds

We will build a small `IntBag` container and use nested classes for iteration, helpers, and filtering.

Step 1: Define a small callback interface

```
interface IntPredicate {
    boolean test(int x);
}
```

This lets us pass "a condition" (behavior) into a method like `countIf(...)`.

Step 2: Outer class + inner cursor (needs outer state)

```
import java.util.Arrays;

public class IntBag {
    private final int[] data;

    public IntBag(int[] data) {
        this.data = data.clone();
    }

    // INNER class: it needs access to 'data'
    public class Cursor {
        private int idx = 0;
        public boolean hasNext() { return idx < data.length; }
        public int next() { return data[idx++]; }
    }

    public Cursor cursor() { return new Cursor(); }

    public int countIf(IntPredicate p) {
        int c = 0;
        for (int x : data) if (p.test(x)) c++;
        return c;
    }

    @Override public String toString() {
        return Arrays.toString(data);
    }
}
```

Why inner? `Cursor` directly reads `data` from the outer object.

Step 3: Static nested helper (does not need outer object)

```
public class IntBag {
    // ... keep previous code ...

    public static class Stats {
        public static int max(int[] a) {
            int m = Integer.MIN_VALUE;
            for (int x : a) if (x > m) m = x;
            return m;
        }
    }
}
```

Why static? `Stats.max` only needs an array; it does not need an `IntBag` instance.

Step 4: Local class and anonymous class (filters)

```java
public class Demo {
    public static void main(String[] args) {
        IntBag bag = new IntBag(new int[]{3, 1, 2, 8, 5});
        System.out.println("Bag = " + bag);

        // INNER class object comes from an outer object
        IntBag.Cursor cur = bag.cursor();
        System.out.print("Iterate: ");
        while (cur.hasNext()) System.out.print(cur.next() + " ");
        System.out.println();

        // LOCAL class (named helper inside a method)
        final int threshold = 3; // explicitly final
        class GreaterThan implements IntPredicate {
            public boolean test(int x) { return x > threshold; }
        }
        System.out.println("count(x > 3) = " + bag.countIf(new GreaterThan()));

        // ANONYMOUS class (one-time implementation)
        int evens = bag.countIf(new IntPredicate() {
            public boolean test(int x) { return x % 2 == 0; }
        });
        System.out.println("count(even) = " + evens);
    }
}
```

Later in Unit 04 you will replace many anonymous classes like the even-check with a lambda, but the idea (passing behavior) remains the same.

12 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Nested classes at a glance* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

13 Practice Check (with brief solutions)

1. Which nested type has an implicit reference to the outer object? **Answer:** inner class.
2. Which nested type is often used for callbacks/listeners? **Answer:** anonymous class.

3. Can a local class access a non-final local variable? **Answer:** no (must be effectively final).

14 Self-Study Practice (with hints)

1. Add a method `sum()` to `IntBag`. Keep it simple.

   Hint: loop over `data`.

2. Modify the demo to count odd numbers using an anonymous class.

   Hint: `x % 2 != 0`.

3. Make a new static nested class `MinMax` that computes both min and max.

   Hint: return an `int[]` of length 2 or create a tiny nested data class.

15 Summary (1 minute)

- Use a **static nested class** when the helper does not need an outer object.
- Use an **inner class** when the helper needs outer state.
- Use a **local class** when the helper is only for one method but benefits from a name.
- Use an **anonymous class** for one-time callbacks (often replaced by lambdas later).

16 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

17 Exit Questions (with sample answers)

Q1. Why do nested classes help encapsulation?

Sample answer: They keep helper types close to where they are used and reduce the public API. This prevents accidental use from unrelated code and improves readability.

Q2. What is the difference between a static nested class and an inner class?

Sample answer: A static nested class does not have an implicit outer reference and can be created without an outer object. An inner class has an implicit reference to an outer instance and can access its instance members.

Q3. What problem do nested classes solve that packages alone do not?

Sample answer: They keep helper behavior tightly scoped to one outer type and reduce unnecessary visibility.